CSA02 C Programming – CO2 Assignment Answers

Assignment 2: Operators, Decisions and Loops

Note: These are model answers based on the uploaded assignment sheets. Replace student-specific details and add your own compiler screenshots, timing measurements and GitHub links before submission.

Q1 – Precedence and discount bug

Multiplication/division precede addition/subtraction; equal-precedence operators associate left-to-right. The original computes gross – (gross×15/100×senior) + gross×5/100. Integer division truncates. net applies Rs 50 reduction after tax. count=count++ + 1 has undefined behaviour because count is modified twice without sequencing; use count++ alone.

Correct: gross=price*qty; discount=senior?gross*15/100:0; taxable=gross-discount; tax=taxable*5/100; total=taxable+tax; net=total>1000?total-50:total. Use integer paise for exact currency.

Q2 – Short-circuit evaluation

&& skips its right operand when left is false; || skips right when left is true. & and | evaluate both operands. Put cheap/selective conditions first if they have no side effects. If server verification must audit every person, call and log it explicitly rather than relying on short-circuit. Nested if can reproduce short-circuit; bitwise operators cannot.

Q3 – Bitwise device register

```
unsigned char switch_on(unsigned char r,int d){return (unsigned char)(r | (1u<<d));}
unsigned char switch_off(unsigned char r,int d){return (unsigned char)(r & ~(1u<<d));}
unsigned char toggle(unsigned char r,int d){return (unsigned char)(r ^ (1u<<d));}
int is_on(unsigned char r,int d){return (r & (1u<<d)) != 0;}
```

After LIGHT and AC on: 00000101. Toggle FAN then switch LIGHT off: 00000110. AC is ON, PUMP is OFF. reg & (1<<dev) tests the target bit; (reg&1)<<dev tests only bit zero. Use unsigned types.

Q4 – Grade ladder and switch

```
char grade(int m){
 if(m<0||m>100)return '?';
 switch(m/10){case 10:case 9:return 'S';case 8:return 'A';case 7:return 'B';case 6:return 'C';case 5:return 'D';default:return '
}
```

Test -1,0,49,50,59,60,69,70,79,80,89,90,99,100,101. Switch on marks/10 groups tens; it cannot directly express arbitrary ranges. Missing break causes fall-through. Lookup table gives O(1) with 11 bytes; ladder can require O(k) comparisons.

Q5 – Prime test

```
int is_prime(unsigned long n){
 if(n<2)return 0;
 for(unsigned long i=2;i<=n/i;i++){checks++;if(n%i==0)return 0;}
 return 1;
}
```

Test divisors only through √n because any larger factor has a matching smaller factor. n/i avoids overflow from i*i. Supplied expected counts: 97→8; 100→1; 7919→87; 1,000,003→999. Sieve: O(N log log N) preprocessing, O(1) per query, O(N) space.

Q6 – Loan schedule

Use while(balance>0). Each month interest=balance×rate; due=balance+interest; payment=min(fixed_payment,due); new balance=due-payment. If payment <= monthly interest, the balance will not decrease and the loop cannot finish; reject terms. Avoid balance!=0 for floating point; use paise or tolerance.

Example Rs 10,000 at 2%, Rs 2,000 repayment: balance after month 1=8,200; month 2=6,364; month 3=4,491.28; month 4=2,581.11; month 5=632.73; final month pays Rs 645.38 including Rs 12.65 interest. Total interest Rs 645.38.

Q7 – Redesign challenge

Recommended redesign: test divisor 2, then odd divisors only while i<=n/i. This exploits factor pairs and avoids even candidates. Time O(√n), space O(1), versus O(n) trial division. Validate both versions on identical inputs, count divisions and record timing. State measured evidence separately; do not invent execution timings.